


Mastering Python Dictionaries



*A Practical Guide
From Fundamentals to Fluency*

"The dictionary is the only place where success comes before work."

— Vidal Sassoon



Contents



I. Introduction to Dictionaries	3
What is a Dictionary?	3
Why Use Dictionaries?	3
II. Creating and Accessing Dictionaries	3
Three Ways to Create a Dictionary	3
Bracket Notation vs. get()	3
III. Modifying Dictionaries	4
Adding, Updating, and Deleting Entries	4
Safe Removal with pop()	4
IV. Dictionary Methods	4
Method Reference Table	4
V. Iterating Over Dictionaries	5
Looping Through Keys, Values, and Items	5
Sorted Iteration	5
VI. Dictionary Comprehensions	5
Syntax and Common Patterns	5
Filtering and Transforming	5
VII. Nested Dictionaries	6
Modeling Hierarchical Data	6
Safe Nested Access	6
VIII. Practical Exercises	6
Five Hands-On Challenges	6



CHAPTER

I**Introduction to Dictionaries**

A dictionary in Python is a mutable, ordered collection of key-value pairs. It is one of the most versatile and widely used data structures in the language. Dictionaries allow you to store and retrieve data efficiently using descriptive keys rather than numeric indices, making your code far more readable and expressive.

Dictionaries are defined using curly braces `{ }`, with each key-value pair separated by a colon. Keys must be immutable types — strings, numbers, or tuples — while values can be any Python object. Since Python 3.7, dictionaries preserve the order in which items are inserted.

Think of a dictionary like a real-world dictionary: you look up a word (the key) and find its definition (the value). This analogy also explains why each key must be unique — just as no dictionary lists the same word twice.

CHAPTER

II**Creating and Accessing****Creating a Dictionary**

```
# Method 1: Curly braces
student = {"name": "Alice", "age": 22, "grade": "A"}

# Method 2: The dict() constructor
student = dict(name="Alice", age=22, grade="A")

# Method 3: From a list of tuples
student = dict([("name", "Alice"), ("age", 22)])

# An empty dictionary
empty = {}
```

Accessing Values

You can retrieve values using square bracket notation or the **get()** method. Brackets raise a **KeyError** when the key is absent; **get()** returns **None** or a specified default instead — making it the safer choice.

```
print(student["name"]) # Alice
print(student.get("gpa", 0.0)) # 0.0 (default)

if "age" in student:
    print(f"Age: {student['age']}")
```

*Prefer **get()** whenever you are uncertain whether a key exists. It prevents unexpected crashes and leads to cleaner, more defensive code.*

CHAPTER

III

Modifying Dictionaries

Because dictionaries are mutable, you can freely add, update, and remove entries after creation. Mastering these operations is essential for working with dynamic data.

```
student = {"name": "Alice", "age": 22}

# Add a new entry
student["grade"] = "A"

# Update an existing value
student["age"] = 23

# Delete an entry
del student["grade"]

# Safely remove and return a value
age = student.pop("age", None)

# Merge in multiple values at once
student.update({"age": 24, "gpa": 3.9})
```

Using **del** on a missing key raises a **KeyError**. Prefer **pop(key, default)** for safe removal: **student.pop("missing", None)** returns **None** instead of crashing.

CHAPTER

IV

Dictionary Methods

Python dictionaries include a rich set of built-in methods. The table below summarizes the ones you will reach for most often.

Method	Description	Example
--------	-------------	---------

<code>keys()</code>	Returns all keys	<code>d.keys()</code>
<code>values()</code>	Returns all values	<code>d.values()</code>
<code>items()</code>	Returns (key, value) tuples	<code>d.items()</code>
<code>get(k, d)</code>	Value or default	<code>d.get("x", 0)</code>
<code>setdefault(k, d)</code>	Get or set a default	<code>d.setdefault("x", [])</code>
<code>update()</code>	Merge another dict in	<code>d.update({...})</code>
<code>pop(k)</code>	Remove; return value	<code>d.pop("x")</code>
<code>clear()</code>	Remove all items	<code>d.clear()</code>
<code>copy()</code>	Shallow copy	<code>d.copy()</code>

CHAPTER

V

Iterating Over Dictionaries

Looping through dictionaries is a fundamental skill. Python provides several elegant ways to iterate over keys, values, or both at once.

```
scores = {"math": 95, "science": 88, "english": 92}

# Keys (default)
for subject in scores:
    print(subject)

# Values only
for score in scores.values():
    print(score)

# Key-value pairs
for subject, score in scores.items():
    print(f"{subject}: {score}")

# Sorted by key
for subject in sorted(scores):
    print(f"{subject}: {scores[subject]}")
```

Calling `.items()` returns a lightweight view object — it does not copy the underlying data. Iteration remains memory-efficient even on very large dictionaries.

CHAPTER

VI

Dictionary Comprehensions

Comprehensions offer a concise, Pythonic way to build new dictionaries from iterables. The general form is: `{key_expr: val_expr for item in iterable if condition}`.

```
# Squares
squares = {x: x**2 for x in range(1, 6)}

# Filter: keep scores >= 90
high = {k: v for k, v in scores.items() if v >= 90}

# Swap keys and values
inverted = {v: k for k, v in scores.items()}

# From two parallel lists
names = ["Alice", "Bob", "Carol"]
ages = [25, 30, 28]
people = {n: a for n, a in zip(names, ages)}
```

*Comprehensions are not merely syntactic sugar. The Python interpreter optimizes them internally, so they often outperform an equivalent **for**-loop.*

CHAPTER

VII

Nested Dictionaries

Dictionaries can contain other dictionaries as values, forming nested structures. This pattern is essential for working with JSON data, configuration files, and any hierarchical record.

```
classroom = {
    "student_1": {
        "name": "Alice",
        "grades": {"math": 95, "science": 88}
    },
    "student_2": {
        "name": "Bob",
        "grades": {"math": 78, "science": 92}
    }
}

# Deep access
alice_math = classroom["student_1"]["grades"]["math"]

# Safe nested access with chained get()
grade = classroom.get("student_3", {}).get("grades", {})
```

CHAPTER

VIII

Practical Exercises

Put your knowledge to the test. Attempt each exercise before consulting any reference material.

Exercise I — Word Counter. Write a function that accepts a string and returns a dictionary mapping each word to its frequency.

Exercise II — Merge & Sum. Write a function that merges two dictionaries; when both share a key, sum their numeric values.

Exercise III — Invert. Swap keys and values. Where multiple keys share the same value, group the original keys into a list.

Exercise IV — Deep Get. Implement `deep_get(d, keys)` to safely navigate nested dictionaries, returning `None` on any missing key.

Exercise V — Top Student. Given a nested dictionary of students and scores, return the name of the student with the highest average.



Congratulations — you have completed this guide. Dictionaries are a cornerstone of Python; practise these exercises and weave dictionaries into your own projects to build true fluency.